



Plataforma de Voluntariado Sustentável

Impacto Social · Sustentabilidade · Conexão

DOCUMENTAÇÃO TÉCNICA DO PROJETO

Disciplina de Engenharia de Software

Versão	1.0 — Maio / 2026
Status	Em Desenvolvimento (MVP)
Repositório	github.com/FranciellyCOding/EcoLink
Framework	Next.js 16 · React 19 · TypeScript
Estilização	Tailwind CSS v4 · shadcn/ui · Radix UI

1. Visão Geral do Projeto

O EcoLink é uma plataforma web de voluntariado sustentável desenvolvida com Next.js 16 e React 19, cujo objetivo central é conectar voluntários a ONGs, empresas com programas de responsabilidade social e projetos comunitários. O sistema promove engajamento cívico e impacto social positivo por meio de uma interface moderna, responsiva e gamificada.

A plataforma está sendo desenvolvida como projeto acadêmico para a disciplina de Engenharia de Software, seguindo princípios de arquitetura de software, levantamento de requisitos, modelagem UML e documentação profissional.

1.1 Declaração de Missão

"Conectar voluntários a instituições comprometidas com o impacto social e a sustentabilidade, reconhecendo suas ações por meio de um sistema de pontos e prêmios que incentiva a participação contínua."

1.2 Objetivos do Sistema

- Conectar voluntários a oportunidades de impacto social e ambiental de forma simples e acessível.
- Permitir que ONGs, empresas e projetos sociais publiquem e gerenciem oportunidades de voluntariado.
- Reconhecer e recompensar o engajamento dos voluntários via sistema de pontos, badges e recompensas externas.
- Fornecer ao administrador da plataforma ferramentas de governança, moderação e análise de impacto.
- Gerar relatórios de impacto social para demonstrar o valor do voluntariado para a sociedade.

1.3 Stakeholders

Stakeholder	Papel	Interesse Principal
Voluntário	Usuário final	Encontrar oportunidades, acumular pontos e certificados
Instituição (ONG/Empresa)	Publicante	Recrutar voluntários e gerenciar ações sociais
Administrador	Operador da plataforma	Moderar, aprovar e gerar relatórios de impacto
Parceiros Externos	Provedores de benefícios	Oferecer descontos/recompensas para voluntários
Equipe de Dev	Desenvolvedores	Implementar, manter e evoluir a plataforma

2. Arquitetura e Stack Tecnológica

O EcoLink é uma aplicação web full-stack construída sobre o ecossistema JavaScript/TypeScript moderno, utilizando a arquitetura de App Router do Next.js 16 com renderização híbrida (Server Components e Client Components).

2.1 Stack Tecnológica

Camada	Tecnologia	Versão / Observação
Frontend Framework	Next.js	v16.2.6 — App Router, SSR/SSG/CSR
UI Library	React	v19 — Server & Client Components
Linguagem	TypeScript	v5.7.3 — tipagem estática completa
Estilização	Tailwind CSS	v4.2 — utility-first CSS
Componentes UI	shadcn/ui + Radix UI	Acessibilidade e consistência visual
Animações	Framer Motion	v12 — transições e micro-interações
Gráficos	Recharts	v2.15 — dashboards e analytics
Formulários	React Hook Form + Zod	Validação tipada e performática
Fontes	Inter + Poppins (Google Fonts)	Hierarquia tipográfica
Analytics	Vercel Analytics	Monitoramento de uso em produção
Ícones	Lucide React	v0.564 — biblioteca de ícones SVG

2.2 Estrutura de Diretórios

O projeto segue a convenção de diretórios do Next.js App Router:

Diretório / Arquivo	Responsabilidade
app/	Rotas da aplicação (App Router do Next.js)
app/page.tsx	Landing page pública (Home)
app/login/page.tsx	Página de autenticação do usuário
app/cadastro/page.tsx	Cadastro de voluntário ou instituição (multi-step)
app/oportunidades/page.tsx	Feed de oportunidades com filtros e busca
app/oportunidades/[id]/page.tsx	Detalhes de oportunidade específica (rota dinâmica)
app/perfil/page.tsx	Perfil do voluntário com histórico, badges e gráficos
app/recompensas/page.tsx	Catálogo de recompensas e ranking de voluntários
app/admin/page.tsx	Dashboard administrativo com métricas e gestão

Diretório / Arquivo	Responsabilidade
app/instituicao/dashboard/page.tsx	Dashboard da instituição com candidatos e analytics
app/configuracoes/page.tsx	Configurações de conta e preferências
app/como-funciona/page.tsx	Página explicativa do funcionamento da plataforma
app/para-ons/page.tsx	Landing page direcionada para instituições
components/landing/	Seções da landing page (Hero, HowItWorks, Gamification...)
components/layout/	Navbar e Footer globais
components/ui/	Biblioteca de componentes reutilizáveis (shadcn/ui)
lib/mock-data.ts	Dados mockados simulando banco de dados
hooks/	Hooks customizados do React
public/	Arquivos estáticos (imagens, favicon)

2.3 Padrões Arquiteturais Adotados

- App Router (Next.js 16): separação entre Server Components (sem estado, SSR/SSG) e Client Components (com estado, interativos).
- Component-Driven Development: cada página é composta por componentes coesos, reutilizáveis e independentes.
- Mock-First Development: dados simulados em lib/mock-data.ts permitem desenvolvimento do frontend independente do backend.
- Colocação de estilos: Tailwind CSS inline, sem folhas de estilo externas por componente.
- Formulários controlados: React Hook Form + Zod garantem validação tipada e feedback imediato ao usuário.

3. Módulos do Sistema

O EcoLink é organizado em seis módulos funcionais principais, cada um com responsabilidades bem delimitadas:

3.1 Módulo de Autenticação e Cadastro

Responsável pelo fluxo de entrada de novos usuários na plataforma. Implementado em `app/cadastro/page.tsx` e `app/login/page.tsx`.

Funcionalidades implementadas no frontend:

- Cadastro em 2 etapas (multi-step form) com animação Framer Motion entre passos.
- Seleção de tipo de conta: Voluntário (campos: nome, e-mail, senha, telefone, cidade) ou Instituição (campos adicionais: CNPJ, tipo de organização, website).
- Toggle de visibilidade de senha (ícone Eye/EyeOff).
- Página de login com redirecionamento mockado para `/oportunidades`.
- Design split-screen: formulário à esquerda, painel visual à direita com gradiente e estatísticas da plataforma.

3.2 Módulo de Oportunidades

Feed principal da plataforma, implementado em `app/oportunidades/page.tsx` e `app/oportunidades/[id]/page.tsx`.

Funcionalidades implementadas no frontend:

- Listagem de oportunidades mockadas com cards visuais (imagem, organização, localização, data, pontos, vagas).
- Busca em tempo real por título e organização.
- Filtro por categoria: Todas, Meio Ambiente, Educação, Saúde, Animais, Social.
- Painel de notificações dropdown com 5 tipos: pontos, oportunidade, confirmação, badge, recompensa.
- Navbar interna com avatar, saldo de pontos e menu mobile responsivo.
- Página de detalhe dinâmica via rota `/oportunidades/[id]`.
- Modelo de dados: id, título, organização, localização, data, horário, pontos, vagas, tags, descrição, requisitos, impacto, categoria.

3.3 Módulo de Perfil e Gamificação

Implementado em `app/perfil/page.tsx` e `app/recompensas/page.tsx`.

Perfil do Voluntário:

- Exibição de: nome, nível, pontos, horas voluntárias, vidas impactadas e streak (dias consecutivos).
- Barra de progresso até próximo nível (calculada como pontos % 500).
- Grade de Badges/Conquistas: 8 badges com ícones, descrições e status (earned/locked).
- Histórico de ações recentes com pontuação por ação.
- Gráfico de evolução mensal de pontos (AreaChart do Recharts).

Sistema de Recompensas:

- 5 níveis de progressão: Semente (0 pts) → Broto (500) → Árvore (2.000) → Floresta (5.000) → Guardião (10.000).
- Catálogo de recompensas com parceiros: Natura, Cinemark, EcoLink, EcoShop, EcoAcademy.

- Categorias de prêmios: Cupons, Entretenimento, Produtos, Certificados, Educação.
- Ranking global de voluntários com posição, avatar, pontos e nível.

3.4 Módulo de Dashboard da Instituição

Implementado em `app/instituicao/dashboard/page.tsx`. Sidebar de navegação com 5 seções: Dashboard, Oportunidades, Candidatos, Analytics e Configurações.

Funcionalidades implementadas no frontend:

- 4 cards de métricas: Oportunidades Ativas, Total de Voluntários, Horas Voluntárias, Candidatos Pendentes.
- Gráfico de evolução mensal de voluntários (AreaChart).
- Listagem de candidaturas recentes com status: pending, approved, rejected.
- Dados mockados: 1.234 voluntários, 15.600 horas, 12.500 árvores plantadas.

3.5 Módulo Administrativo

Implementado em `app/admin/page.tsx`. Sidebar com 7 seções: Dashboard, Usuários, Instituições, Oportunidades, Sistema de Pontos, Relatórios, Configurações.

Funcionalidades implementadas no frontend:

- Métricas globais: 15.420 usuários, 234 instituições, 1.567 oportunidades, 45.678 ações, 2.345.000 pontos distribuídos.
- Gráfico de crescimento de usuários mensais (AreaChart) e distribuição por categoria (PieChart).
- Tabela de usuários recentes com tipo (Voluntário/Instituição) e status (active/pending).
- Tabela de denúncias recentes com tipo e alvo.
- Ações disponíveis: ver, banir, aprovar por usuário/instituição.

3.6 Módulo de Landing Page

Implementado em `app/page.tsx` com 9 seções compostas por componentes independentes:

Componente	Conteúdo
HeroSection	Chamada principal, CTA de cadastro e login, estatísticas animadas
HowItWorksSection	3 passos: cadastre-se, escolha, participe e ganhe pontos
BenefitsSection	Benefícios para voluntários e para instituições
ImpactSection	Métricas de impacto social da plataforma
GamificationSection	Explicação do sistema de pontos, badges e níveis
TestimonialsSection	Depoimentos de voluntários e gestores de ONGs
PartnersSection	Logos dos parceiros externos (Natura, Boticário, Itaú, etc.)
CTASection	Chamada final para cadastro
Footer	Links, redes sociais e informações legais

4. Levantamento de Requisitos (Síntese)

O levantamento completo de requisitos está detalhado no documento separado (Requisitos_Voluntariado_Sustentavel.docx). Esta seção apresenta a síntese por módulo e os principais requisitos mapeados ao código atual.

4.1 Resumo Quantitativo

Categoria	Total	Impl. no MVP	Pendente Backend
Requisitos Funcionais (RF)	31	18	13
Requisitos Não Funcionais (RNF)	17	9	8
Regras de Negócio (RN)	10	4	6
Requisitos Extras Sugeridos (RE)	8	2	6
TOTAL	66	33	33

4.2 Requisitos Implementados no Frontend (MVP)

- RF-01/02: Cadastro de voluntário e instituição com multi-step form e validação.
- RF-03: Login com redirecionamento por tipo de conta.
- RF-06/07: Perfil do voluntário com dados, badges, histórico e edição via configurações.
- RF-08: Perfil da instituição via dashboard.
- RF-09/11/12: Publicação, listagem, busca e visualização de oportunidades.
- RF-21/22: Exibição de saldo de pontos e extrato na interface.
- RF-23/24: Catálogo de recompensas e ranking de voluntários.
- RF-27/28/29: Painel administrativo com métricas, listagem e ações de moderação.

4.3 Requisitos Pendentes (Dependem de Backend)

- RF-14/15/16: Inscrição, cancelamento e gestão de candidaturas (requer API + banco de dados).
- RF-17: Notificações em tempo real (requer WebSocket ou polling).
- RF-19/20: Confirmação de presença e geração de certificado PDF.
- RF-25/26: Sistema de avaliação mútua voluntário ↔ instituição.
- RF-31: Exportação de relatórios em PDF e CSV.
- RNF-04/06/07: Criptografia, proteção contra ataques e conformidade com LGPD.

5. Modelagem UML

5.1 Diagrama de Caso de Uso — Visão Geral

O diagrama de caso de uso mapeia as interações entre os três atores do sistema (Voluntário, Instituição e Administrador) e os casos de uso disponíveis na plataforma. Os principais relacionamentos são:

Voluntário:

- Criar conta → <<include>> Estar logado
- Fazer login → <<include>> Estar logado
- Visualizar oportunidades → <<extend>> Estar logado (opcional)
- Candidatar-se a vaga → <<include>> Estar logado
- Ganhar pontos → <<include>> Validar participação pela instituição
- Trocar pontos por prêmios → <<include>> Verificar saldo de pontos → <<include>> Registrar resgate

Instituição:

- Criar conta institucional + Fazer login → Estar logada
- Cadastrar oportunidade → <<include>> Informar dados (Descrição, Local, Data, Vagas, Área)
- Confirmar participação do voluntário → <<include>> Sistema de Pontos

Administrador:

- Gerenciar usuários, instituições, oportunidades, sistema de pontos
- Gerar relatórios de impacto social

5.2 Diagrama de Atividade — Resumo dos Fluxos

O diagrama de atividade (arquivo: Diagrama_de_atividades_-_Ecolink.drawio e diagrama_atividade_ecolink.puml) modela os 8 fluxos principais do sistema em 3 swim lanes:

#	Fluxo	Descrição
①	Cadastro e Autenticação	Decisão 'Tem conta?' → Criar conta + Verificar e-mail OU Fazer login → Sessão iniciada
②	Publicação de Oportunidade	Instituição aprovada cadastra oportunidade com todos os dados obrigatórios e publica
③	Busca e Candidatura	Voluntário filtra oportunidades → verifica vaga → candidata-se → acompanha status
④	Confirmação e Pontos	Instituição valida presença → calcula pontos por carga horária → gera certificado PDF
⑤	Troca de Pontos	Voluntário verifica saldo (mín. 100 pts) → seleciona prêmio → resgate registrado
⑥	Aprovação de Instituições	Admin analisa cadastro → aprova (libera acesso) ou reprova (notifica instituição)
⑦	Moderação e Relatórios	Admin gerencia usuários, modera conteúdo e gera relatórios de impacto social

#	Fluxo	Descrição
⑧	Avaliação Mútua	Voluntário avalia instituição (1-5★) e instituição avalia voluntário após a atividade

5.3 Modelo de Dados (Mock)

O arquivo lib/mock-data.ts define as entidades e seus atributos, simulando o contrato de API esperado:

Entidade	Atributos Principais
Oportunidade	id, title, organization, organizationLogo, image, location, date, time, points, participants, maxParticipants, tags[], description, requirements[], impact, category
Usuário (Voluntário)	id, name, email, avatar, level, points, hoursVolunteered, actionsCompleted, livesImpacted, streak, joinedDate, badges[], recentActions[], monthlyProgress[]
Badge	id, name, icon, description, earned
Recompensa	id, title, description, points, category, image, available, partner
Ranking	position, name, avatar, points, level
Instituição	id, name, logo, description, email, totalOpportunities, activeOpportunities, totalVolunteers, totalHours, impactMetrics{}, recentApplications[], monthlyVolunteers[]
Candidatura	id, volunteer, opportunity, date, status (pending approved rejected)
AdminStats	totalUsers, totalInstitutions, totalOpportunities, totalActionsCompleted, totalPointsDistributed, activeUsers, pendingApprovals, reportedContent, userGrowth[], categoryDistribution[]

6. Definição do MVP (Produto Mínimo Viável)

O MVP do EcoLink contempla todas as funcionalidades essenciais para validar a proposta de valor da plataforma: conectar voluntários a oportunidades, recompensar engajamento e fornecer gestão às instituições. O frontend do MVP está implementado; as funcionalidades de backend (persistência, autenticação real, API) compõem a próxima fase.

6.1 Escopo do MVP — Frontend (Implementado)

#	Funcionalidade	Página / Rota	Status
1	Landing page pública com apresentação da plataforma	/	☑ Implementado
2	Página explicativa (Como Funciona + Para ONGs)	/como-funciona · /para-ongs	☑ Implementado
3	Cadastro multi-step voluntário e instituição	/cadastro	☑ Implementado
4	Login com redirecionamento por tipo de conta	/login	☑ Implementado
5	Feed de oportunidades com busca e filtros	/oportunidades	☑ Implementado
6	Detalhe de oportunidade com candidatura (UI)	/oportunidades/[id]	☑ Implementado
7	Perfil do voluntário com badges, histórico e gráfico	/perfil	☑ Implementado
8	Sistema de recompensas e ranking	/recompensas	☑ Implementado
9	Configurações de conta	/configuracoes	☑ Implementado
10	Dashboard da instituição com candidatos e analytics	/instituicao/dashboard	☑ Implementado
11	Painel administrativo com métricas e moderação	/admin	☑ Implementado

6.2 Próxima Fase — Backend e Integrações

- Implementação da API RESTful (Node.js/Next.js API Routes ou backend dedicado).
- Banco de dados relacional (PostgreSQL) com Prisma ORM para persistência.
- Autenticação real com JWT + NextAuth.js (suporte a OAuth Google/Facebook).
- Sistema de e-mails transacionais (confirmação de cadastro, notificações de candidatura).
- Upload de imagens para perfis e oportunidades (Amazon S3 ou Cloudinary).
- Geração de certificados PDF no servidor (Puppeteer ou PDFKit).
- Integração com parceiros externos para resgate de pontos.
- Testes unitários e de integração (Jest + Testing Library).

7. Qualidade e Requisitos Não Funcionais

7.1 Desempenho

- Tempo de resposta alvo: ≤ 2 segundos para 500 usuários simultâneos.
- Next.js com SSR/SSG reduz Time to First Byte (TTFB) e melhora SEO.
- Vercel Edge Network garante CDN global e baixa latência.
- Recharts renderizado no cliente para gráficos sem overhead no servidor.

7.2 Segurança (Roadmap)

- Autenticação: JWT com refresh token, expiração de 1 hora, rotação automática.
- RBAC (Role-Based Access Control): voluntário, instituição, administrador.
- HTTPS obrigatório em produção (Vercel fornece TLS automático).
- Sanitização de inputs e proteção contra XSS com React DOM (escape automático).
- Conformidade LGPD: consentimento explícito no cadastro, política de privacidade, exclusão de conta.

7.3 Acessibilidade

- Radix UI e shadcn/ui garantem componentes com semântica ARIA correta.
- Navegação por teclado suportada em menus, modais e formulários.
- Contraste de cores verificado (WCAG 2.1 AA) nas paletas verde e azul da aplicação.
- Fontes Inter e Poppins com excelente legibilidade em diversas resoluções.

7.4 Manutenibilidade

- TypeScript em 100% do código garante tipagem e detecção de erros em tempo de desenvolvimento.
- Componentes shadcn/ui com padrão de composição (Compound Components).
- Mock data centralizada em lib/mock-data.ts facilita substituição por API real.
- Tailwind CSS com classes semânticas e sem CSS customizado disperso.

7.5 Portabilidade e Compatibilidade

- Interface responsiva (mobile-first) testada em smartphones, tablets e desktops.
- Compatível com Chrome, Firefox, Safari e Edge (últimas 2 versões).
- Deploy automático na Vercel — zero configuração de servidor.
- API RESTful planejada para suportar app mobile nativo no futuro.

8. Planejamento e Roadmap

8.1 Fases do Projeto

Fase	Nome	Entregas	Status
1	Levantamento	Requisitos funcionais/não funcionais, regras de negócio, diagramas UML (caso de uso, atividade)	✓ Concluído
2	Prototipação	Wireframes e design das telas no Figma / v0.app	✓ Concluído
3	Frontend MVP	Implementação de todas as rotas e componentes com dados mockados	✓ Concluído
4	Backend API	API RESTful, banco de dados PostgreSQL, autenticação JWT	🔄 Em andamento
5	Integração	Conexão frontend ↔ backend, testes E2E, ajustes de UX	⌚ Planejado
6	Lançamento	Deploy em produção, onboarding de ONGs piloto, monitoramento	⌚ Planejado

8.2 Backlog de Funcionalidades — Prioridades

Funcionalidade	Prioridade	Observação
API de autenticação real (JWT + NextAuth)	● Alta	Base para todas as funcionalidades autenticadas
CRUD de oportunidades com banco de dados	● Alta	Core do produto — publicação e gestão
Sistema real de candidaturas	● Alta	Permite o fluxo completo voluntário ↔ instituição
Confirmação de presença e pontuação real	● Alta	Ativa o motor de gamificação
E-mails transacionais	🌀 Média	Confirmação de cadastro, aceite de candidatura
Geração de certificado PDF	🌀 Média	Diferencial de valor para os voluntários
Dashboard analítico real	🌀 Média	Métricas reais de impacto social
Integração com parceiros de recompensas	🌀 Média	Fecha o ciclo de gamificação
Notificações em tempo real (WebSocket)	🌀 Média	Melhora engajamento e UX
App mobile nativo (React Native)	● Baixa	Expansão futura da plataforma
Mapa de impacto geográfico	● Baixa	Visualização de alcance territorial

Funcionalidade	Prioridade	Observação
Mapeamento aos ODS da ONU	 Baixa	Posicionamento e relatório de impacto

9. Considerações Finais

O EcoLink representa uma solução tecnológica contemporânea e socialmente relevante para o problema da baixa retenção e engajamento no voluntariado. A utilização de gamificação, design moderno e uma arquitetura escalável posicionam a plataforma de forma competitiva no mercado de aplicações de impacto social.

Do ponto de vista técnico, a escolha do Next.js 16 com React 19 e TypeScript garante performance, tipagem segura e uma excelente experiência de desenvolvimento. A integração com shadcn/ui e Radix UI assegura acessibilidade nativa, enquanto o Tailwind CSS v4 proporciona estilização consistente e manutenível.

O MVP de frontend entregue demonstra a viabilidade e a completude visual da proposta, servindo como base sólida para as próximas iterações que incluirão o backend, a persistência de dados e as integrações externas.

Esta documentação segue as boas práticas da Engenharia de Software e está alinhada com os artefatos produzidos ao longo do projeto: levantamento de requisitos, diagramas UML (caso de uso e atividade) e planejamento de MVP. Recomenda-se que seja atualizada a cada sprint ou iteração de desenvolvimento.